

```

#include #include #include #include using namespace std; class ParserGramatica2 { private:
string input; size_t pos; void skipWhitespace() { while (pos < input.length() &&
isspace(input[pos])) pos++; } bool match(char expected) { skipWhitespace(); if (pos <
input.length() && input[pos] == expected) { pos++; return true; } return false; } bool numero() {
skipWhitespace(); if (pos < input.length() && isdigit(input[pos])) { while (pos < input.length() &&
isdigit(input[pos])) pos++; return true; } return false; } bool factor() { size_t old_pos = pos; if
(match('(')) { if (expr()) { if (match('')) return true; } pos = old_pos; return false; } pos = old_pos; if
(numero()) return true; return false; } bool term() { if (!factor()) return false; while (true) { size_t
old_pos = pos; if (match('*') || match('/')) { if (!factor()) { pos = old_pos; break; } } else break; }
return true; } bool expr() { if (!term()) return false; while (true) { size_t old_pos = pos; if
(match('+') || match('-')) { if (!term()) { pos = old_pos; break; } } else break; } return true; } public:
bool evaluar(const string& str) { input = str; pos = 0; bool esValida = expr(); skipWhitespace();
return esValida && (pos == input.length()); } }; int main() { ParserGramatica2 parser; vector
sentencias = { "3+4*5-6", "7*(6-8)-6", "(7*9)-5+(4*3)" }; for (const string& s : sentencias) { if
(parser.evaluar(s)) { cout << s << " -> PERTENECE\n"; } else { cout << s << " -> NO
PERTENECE\n"; } } return 0; }

```